

Advanced Higher Computing Science

Software Design and Development using Python



Booklet 3 – Standard Algorithms

Section 3 – Standard Algorithms	2
Reminder	2
Bubble Sort on a 1-D Array	2
Description	2
Design of Bubble Sort on a 1-D Array	3
Implementation and Testing of Bubble Sort on a 1-D Array in Python 3	4
Bubble Sort on a 1-D Array - Exercise	5
Insertion Sort on a 1-D Array	6
Description	6
Design of Insert Sort on a 1-D Array	6
Implementation and Testing of Bubble Sort on a 1-D Array in Python 3	7
Insertion Sort on a 1-D Array - Exercise	8
Binary Search on a 1-D Array	9
Description	9
Design of Binary Search Sort on a 1-D Array	9
Implementation and Testing of Bubble Sort on a 1-D Array in Python 3	10
Binary Search on a 1-D Array – Exercise	11
Standard Algorithms with an Array of Objects	12
Description	12
Standard Algorithms with an Array of Objects – Exercise	14

Section 3 – Standard Algorithms

Reminder

At National 5 Computing Science, the following algorithms are covered:

- Input Validation
- Traversing a 1-D Array
- Running Total within a 1-D Array

At Higher Computing Science, the following algorithms are covered:

- Linear Search within Parallel 1-D Arrays or Array of Records
- Count Occurrences within Parallel 1-D Arrays or Array of Records
- Finding Maximum/ Minimum within Parallel 1-D Arrays or Array of Records

For Advanced Higher Computing Science, it is important to recognise that all the above can and should be used. Of particular importance is the use of Input Validation. All inputs at Advanced Higher level should be validated.

Bubble Sort on a 1-D Array

Description

The Bubble Sort Standard Algorithm works by repeatedly going through the array, swapping the adjacent elements if they are in wrong order. The first time the algorithm goes through the array, the largest item moves (or bubbles) to the end of the array. The next time, the second highest bubbles to the second from the end of the array. This repeats until there are no swaps, or the array has been passed through as many times as there are items.

A swap is carried out by holding one of the elements to be swapped in a temporary variable. The other element then overwrites the original, with the temporary variable then overwriting the other element.

Implementing a Bubble Sort on a 1-D Array in SQA-RL

Each time through the array is known as a pass. It is possible that the algorithm is supplied an already sorted (or partially sorted) set of data, so a conditional loop is used to end the algorithm before unnecessary and inefficient passes are made.

```

Line 1    FUNCTION binarySort RETURNS ARRAY OF INTEGER
Line 2        SET n to len(nA)
Line 3        SET swapped TO True
Line 4        WHILE swapped AND n >= 0 DO
Line 5            FOR i FROM 0 TO n-1 DO
Line 6                IF nA[i] > nA[i+1] THEN
Line 7                    temp = nA[i]
Line 8                    nA[i] = nA[i+1]
Line 9                    nA[i] = temp
Line 10                SET swapped TO True
Line 11            END IF
Line 12        SET n TO n-1
Line 13    END FOR
Line 14    END WHILE
Line 15    RETURN nA
Line 16    END FUNCTION

Line 17    DECLARE numArray INITIALLY [ 23,546,2,3214,456,234,321 ]
Line 18    SET numArray TO binarySearch (numArray)
Line 19    SEND numArray TO DISPLAY

```

Line 2 sets n to be the size of the array. Line 3 sets swapped to True, without which the conditional loop in line 4 will not start.

Lines 6 to 11 compare each element with the next one in the array and, if the first is larger than the second, will do the swap (lines 7-9). This will also switch swapped back to True, so the algorithm knows that at least one swap has taken place during this pass.

Line 12 is used to decrement n, as the last item in the array will now be the largest number, so there is no need to check the last position.¹

¹ <https://www.youtube.com/watch?v=9I2oOAr2okY>

Implementation and Testing of Bubble Sort on a 1-D Array in Python 3

The Python 3 code shown on the next page shows how the Bubble Sort can be tested, by displaying the unsorted array, carrying out the sort, then displaying the sorted array.

Note that this version of the algorithm uses the same array name (numArray) for both the original unsorted array and the sorted array. In short, the algorithm overwrites the original array with the sorted data. If the program requires to still use the original unsorted array, it is wise to return the sorted data to a different array.

```
def bubbleSort(nA):

    n = len(nA)
    swapped = True

    while swapped and n>=0:
        swapped = False
        for i in range(0,n-1): #changed from algorithm (n-2)
            if nA[i] > nA[i+1]:
                temp = nA[i]
                nA[i] = nA [i+1]
                nA[i+1] = temp
                swapped = True

        n = n-1
    return (nA)

numArray = [23,546,2,3214,456,234,321]
print (numArray)
numArray = bubbleSort(numArray) #if you want to keep the original array, assign to a
new array at this point
print (numArray)
```

Program 15 – Bubble Sort on 1D Array

Points to note

- The Bubble Sort uses an external conditional loop, with an internal fixed loop.
- n decrements each pass through the conditional array, so that the algorithm only needs to check the remaining items, not the sorted list that starts to exist at the end after a few passes.
- For evidence, **always** provide the unsorted array and the sorted array.



Bubble Sort on a 1-D Array - Exercise

1. Using the Bubble Sort program supplied, expand the original data set to ensure that the array is scalable.
2. What happens if the data provided to the Bubble Sort function is an array of string?

Insertion Sort on a 1-D Array

Description

The Insertion Sort Standard Algorithm works similarly to how a deck of cards would be sorted. When a card is found that is out of order, it is removed from its current location. A gap is created at the correct location in the list, with all cards after that moving on one location, until they get to the gap created by the removal of the card being placed. That card is then placed into the gap.²

Implementing an Insertion Sort on a 1-D Array in SQA-RL

```

Line 1    FUNCTION insertionSort(nA) RETURNS ARRAY OF INTEGER
Line 2        SET value TO 0
Line 3        SET index TO 0
Line 4        FOR i FROM 1 TO len(nA)
Line 5            SET value TO nA[i]
Line 6            SET index = i
Line 7            WHILE index > 0 AND value < nA[index - 1] DO
Line 8                nA[index] = nA[index-1]
Line 9                index = index - 1
Line 10           END WHILE
Line 11        nA[index] = value
Line 12    END FOR
Line 13    RETURN nA
Line 14    END FUNCTION

Line 15    DECLARE numArray INITIALLY [ 23,546,2,3214,456,234,321 ]
Line 16    SET numArray TO binarySearch (numArray)
Line 17    SEND numArray TO DISPLAY
  
```

The swapping is slightly hidden in the Insertion Sort algorithm, as it holds the value to be checked in the value variable, which is then checked to see where it belongs in the list. Once known, Lines 8 to 10 then move the remaining items one place down the list, before line 11 puts the value in its correct place.

² <https://www.youtube.com/watch?v=kCku5BLI3fo> (note simplified version of swap in the video)

Implementation and Testing of Bubble Sort on a 1-D Array in Python 3

The Python 3 code shown on the next page shows how the Insertion Sort can be tested, by displaying the unsorted array, carrying out the sort, then displaying the sorted array.

Note that this version of the algorithm uses a different array name (numArray and sortedArray) for both the original unsorted array and the sorted array. In short, the algorithm keeps the original unsorted array and provides a brand new sorted array.

```
def insertionSort(nA):
    value = 0
    index = 0
    for i in range (1, len(nA)): #changed from algorithm (len(nA)-1)
        value = nA[i]
        index = i
        while index > 0 and value < nA[index-1]:
            nA[index] = nA[index-1]
            index = index - 1
        nA[index] = value
    return (nA)

numArray = [23,546,2,3214,456,234,321 ]
sortedArray = insertionSort(numArray)
print (sortedArray)
```

Program 16 – Insertion Sort on 1D Array

Points to note:

- The Insertion Sort uses an external fixed loop, with an internal conditional loop. This is the opposite way round to the Bubble Sort algorithm.
- For evidence, **always** provide the unsorted array and the sorted array.



Insertion Sort on a 1-D Array - Exercise

1. Using the Insertion Sort program supplied, expand the original data set to ensure that the array is scalable.
2. What happens if the data provided to the Insertion Sort function is an array of string?

Binary Search on a 1-D Array

Description

Unlike the Linear Search learned at Higher Computing Science, the Binary Search **requires a sorted data set to perform properly**. This is due to how it operates by repeatedly going to the mid-point of the list and checking if the value it is looking for is higher or lower than the mid-point. If lower, it does the same again, with the lower half of the list. If bigger, it does the same with the upper half of the list.

This process repeats until either the search item is found, or the list cannot be split in half any further.³

Because of this, it is important to run either a Bubble Sort or an Insertion Sort on the data set before conducting the Binary Search.

Implementing a Binary Search Sort on a 1-D Array in SQA-RL

```

Line 1    <Bubble Sort Function>
Line 2
Line 3    FUNCTION insertionSort(nA, target) RETURNS INTEGER, STRING
Line 4        SET low TO 0
Line 5        SET high TO len(nA)
Line 6        SET mid TO 0
Line 7        SET found TO False
Line 8        SET position to -1

Line 9        WHILE found = False AND low <= high
Line 10            SET mid TO int(low + high)/2
Line 11            IF target = nA[mid] THEN
Line 12                SET position TO mid
Line 13                SET Found TO True
Line 14            ELSE IF target >= nA[mid] THEN
Line 15                SET low TO mid + 1
Line 16            ELSE
Line 17                SET high TO mid - 1
Line 18            END IF
Line 19        END WHILE
Line 20        RETURN position
Line 21
Line 22    DECLARE numArray INITIALLY [ 23,546,2,3214,456,234,321 ]
Line 23    SET numArray TO bubbleSort(numArray)
Line 24    SET posFound TO insertionSort(numArray)

```

³ <https://www.youtube.com/watch?v=DRsJ8sA9xzc>

Implementation and Testing of Binary Search on a 1-D Array in Python 3

The Python 3 code shown below gives a full worked example of how the Bubble Sort algorithm is applied to the array, with the Binary Search then conducted on the sorted array.

```
def bubbleSort(nA):

    n = len(nA)
    swapped = True

    while swapped and n>=0:
        swapped = False
        for i in range(0,n-1): #changed from algorithm (n-2)
            if nA[i] > nA[i+1]:
                temp = numArray[i]
                nA[i] = nA [i+1]
                nA[i+1] = temp
                swapped = True
        n = n-1
    return nA

def binarySearch(nA, target):
    low = 0
    high = len(nA)-1
    mid = 0
    found = False
    position = -1

    while not(found) and low <= high:
        mid = int((low + high)/2)
        if target == nA[mid]:
            position = mid
            found = True
        elif target >= nA[mid]:
            low = mid + 1
        else:
            high = mid - 1

    return position

numArray =
[23,546,2,3214,456,234,3214,4576,4657,324,321,4658,765,324,6548,435,54687,32,46857,87
659,435,486,87690,5]
numArray = bubbleSort(numArray)
print (numArray)
posFound = binarySearch(numArray,int(input("Please enter value to find: ")))
print (posFound)
```



Points to note:

- An Insertion Sort could have been used just as meaningfully instead of the Bubble Sort.
- The first part of the algorithm, before the loop starts, sets the start and end points of the initial array.
- Setting `position` to be -1 serves as a marker that, if -1 is returned to the top-level program, then a selection can be used to identify that the item being searched for was not found in the list.
- For evidence, **always** provide the unsorted array and the sorted array.

Binary Search on a 1-D Array – Exercise

1. Using the Binary Search program supplied, expand the original data set to ensure that the array is scalable.
2. What happens if the data provided to the Binary Search function is an array of string?
3. Add a corresponding Parallel 1-D Array of string items and use this to display the results of a successful search.
4. Adapt the Binary Search algorithm to account for possibly having multiple equal data items in the array.

Standard Algorithms with an Array of Objects

Description

The three algorithms for Advanced Higher Computing Science, when applied to an Array of Objects will work in the same way as for a 1-D Array, with the following adaptations

- It is likely the Array of Objects may be read in from a CSV or TXT file (or from an external database server)
- Dot notation will need to be used to access the data in each property within the Object's class.

The following program shows how two of the algorithms can be used with file handling to read in an Array of Objects⁴, perform a sort and then search for a specific item.

```
class Car():
#define the structure and content of the City Record
    def __init__( self, car_id,car_make,car_model,car_colour,car_year,car_serviced ):
        self.car_id = car_id #String
        self.car_make = car_make #String
        self.car_model = car_model #String
        self.car_colour = car_colour #String
        self.car_year = car_year #String
        self.car_serviced = car_serviced #String

def readFileToArrayOfObjects():

    cArray = []
    entireFile = open ( "carData.csv" , "r" )
    #this file needs to be in the same folder as the program
    linesArray = entireFile.read().splitlines()
    entireFile.close()
    for line in linesArray:
        lineSplit = line.split( "," )
        currentCar = Car(lineSplit[0], lineSplit[1],
lineSplit[2],lineSplit[3],lineSplit[4],lineSplit[5])
        cArray.append(currentCar)
    return cArray
```

continued on next page

⁴ Technically, this is for an Array of Records, as the Class definition does not have any methods, but the same principles apply!

```
def bubbleSortArrayOfObjects(cArray):
    n = len(cArray)
    swapped = True

    while swapped and n>=0:
        swapped = False
        for i in range(0,n-1): #changed from algorithm (n-2)
            if cArray[i].car_make > cArray[i+1].car_make:
                temp = cArray[i]
                cArray[i] = cArray [i+1]
                cArray[i+1] = temp
                swapped = True
        n = n-1
    return cArray

def binarySearch(cA, target):
    low = 0
    high = len(cA)-1
    mid = 0
    found = False
    position = -1

    while not(found) and low <= high:
        mid = int((low + high)/2)
        if target == cA[mid].car_make:
            position = mid
            found = True
        elif target >= cA[mid].car_make:
            low = mid + 1
        else:
            high = mid - 1

    return position

carArray = readFileToArrayOfObjects()

print ("*Unsorted Array*")
for cars in range (0,len(carArray)):
    print (carArray[cars].car_make)

sortedArray = bubbleSortArrayOfObjects(carArray)

print ("*Unsorted Array*")
for cars in range (0,len(carArray)):
    print (carArray[cars].car_make)
```

continued on next page

```
target = str(input("What make of car are you looking for? "))
position = binarySearch(sortedArray, target)
if position == -1:
    print ("Sorry, we don't have any of that make.")
else:
    print ("Yes! We have that make of car!")
```

Standard Algorithms with an Array of Objects – Exercise

1. Adapt the car program to allow the user to search for
 - a. The make of car
 - b. The year
2. Consider and try the following – is it possible to have a standard algorithm as a method of a class? For example, in the Event class, there is a string array of participants. Could this class have a method called sortParticipants that provides an alphabetical list of the participants? How would this work?